Controlling Consequential Transitions

An Execution Contract and Architecture-Review Method for Agent Workflows

Author and collaboration
Brad Pierce in collaboration with an AI language model.

Contribution statement
Brad Pierce developed the central formulation and retains final editorial responsibility. AI assisted in preparing the paper. This collaboration does not imply endorsement by any organization.

Abstract

“Agents Act. Systems Remember. Rules Authorize. People Govern.” separates productive work, trusted evidence, authorization, and human decision points [1]. This paper asks what execution contract makes those separations enforceable in software. It treats the consequential state transition as the unit of control: evidence is gathered, policy evaluated, scoped authority granted, effects executed, outcomes observed, and authoritative state committed under one reconstructable identity. Existing workflow, policy, provenance, event-history, human-task, and agent-checkpointing systems already provide most of the required primitives. In the systems reviewed here, however, the complete evidence → policy → authority → effect → observation → commit relation is not first-class across product boundaries. The paper derives fourteen requirements and a reference architecture that make this relation explicit and testable, with bounded authority, enforcement at the real effect boundary, independent observation, and reconstructable history. The contract can be realized by composition, a protocol/profile, or an integrated runtime. This is a design sketch rather than an implementation result; its failure-oriented acceptance tests define the proposed validation surface.

1. Recognize the problem

Many stacks already contain a workflow engine, agent framework, policy check, audit log, approval mechanism, and a collection of external tools. Yet the path from “the agent proposes

something” to “the world changed” may still depend on application conventions that are hard to inspect and easy to bypass.

The problem often shows up through small engineering smells.

A workflow reaches an “approved” node, but there is no durable object that says who may do exactly what, to which resource, under what conditions, until when.

An agent or worker reports that a test passed, and that same report is accepted as evidence for the next increase in authority.

A human approval exists, but it is detached from the exact artifact, object version, environment, or target state that the person reviewed.

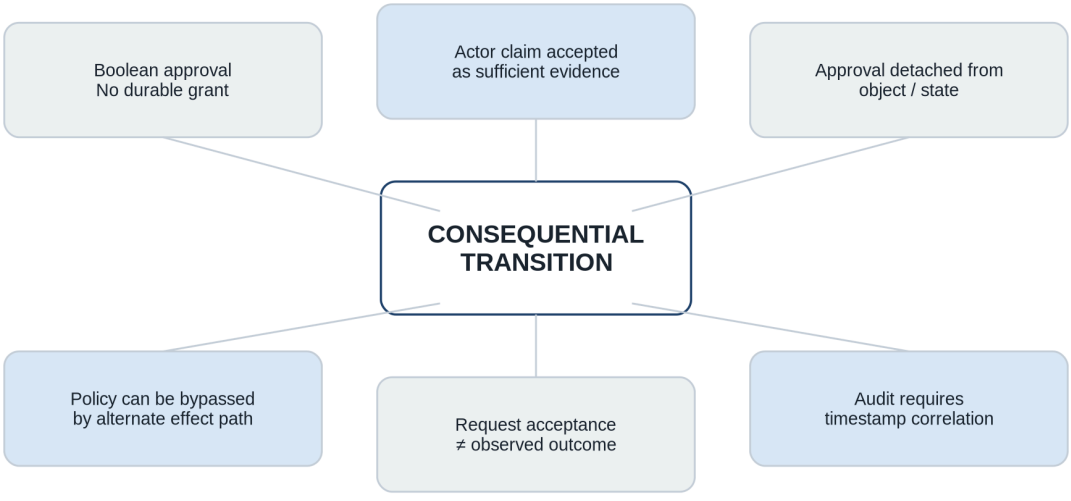
A policy engine returns deny, yet an alternate API, administrator credential, or legacy path can still perform the consequential effect.

An external API accepts a request, and the workflow immediately commits the intended state without independently establishing what the target actually did.

A later audit can reconstruct the incident only by correlating timestamps across several logs and guessing that particular records refer to the same transition.

These are different symptoms of the same missing structure: the consequential transition is not represented explicitly enough.

Figure 1. Six signs the consequential transition is still implicit



These are symptoms of missing transition semantics — not defects in any particular product.

A useful engineering rule of thumb is:

Don’t audit the agent. Audit the transition.

The agent may plan, reason, retry, search alternatives, call low-risk tools, or even be replaced by a conventional service. The transition is where permission, external side effects, and authoritative state meet.

The earlier paper supplies the architectural premise: agents act; trusted systems remember; rules authorize; people govern [1]. It also states one integrity condition: an actor receiving new authority must not be able to manufacture all the evidence sufficient to receive it.

Here, those claims are implementation constraints.

The practical question is:

What execution contract would make those constraints enforceable rather than merely conventional?

What this paper does not require

Implementing this architecture does not require:
• replacing the current workflow engine;
• replacing the current policy engine;
• centralizing all evidence in one database;
• putting humans in every loop;
• cryptographically signing every low-risk event;
• adopting the conceptual object names used in this paper; or
• buying or building a new monolithic platform.

The contract may be distributed across several systems. What matters is whether the semantics are explicit, enforceable, and reconstructable.

Make the stronger contract earn its complexity

A stronger execution contract should beat simpler fixes, not just describe a cleaner model. Three matched-effort baselines should be tried first:

1. Add a stable transition_id and a structured causal log to the existing stack, without changing authority semantics.

2. Keep the existing workflow and policy engines, but mint a short-lived, audience-bound grant and require the real execution gateway to validate it.
3. Implement the same seven transition semantics as disciplined application composition inside the current workflow stack, without introducing a new runtime or protocol profile.

The proposal is weakened if one of these baselines provides comparable bypass resistance, causal reconstructability, ambiguous-outcome handling, and operational clarity with materially less machinery. The integration-gap thesis is defeated if an existing runtime already makes the full evidence → policy → authority → effect → observation → commit relation first-class and enforces it end to end without bespoke application conventions.

A pilot also has to expose the costs of the stronger contract. Centralized authority or evidence services can become latency or availability bottlenecks. Evidence invalidation and revocation can race with execution. Cross-system partial failure can leave the execution system internally consistent while the external world is uncertain. A useful comparison should measure added latency, bypass paths, failure containment, incident-reconstruction effort, and developer burden as well as correctness.

2. Control the transition

The unit we need to control is not an agent session or an LLM invocation. It is a consequential state transition.

A canonical transition has seven stages:

Propose → Gather Evidence → Evaluate → Authorize → Execute → Observe → Commit

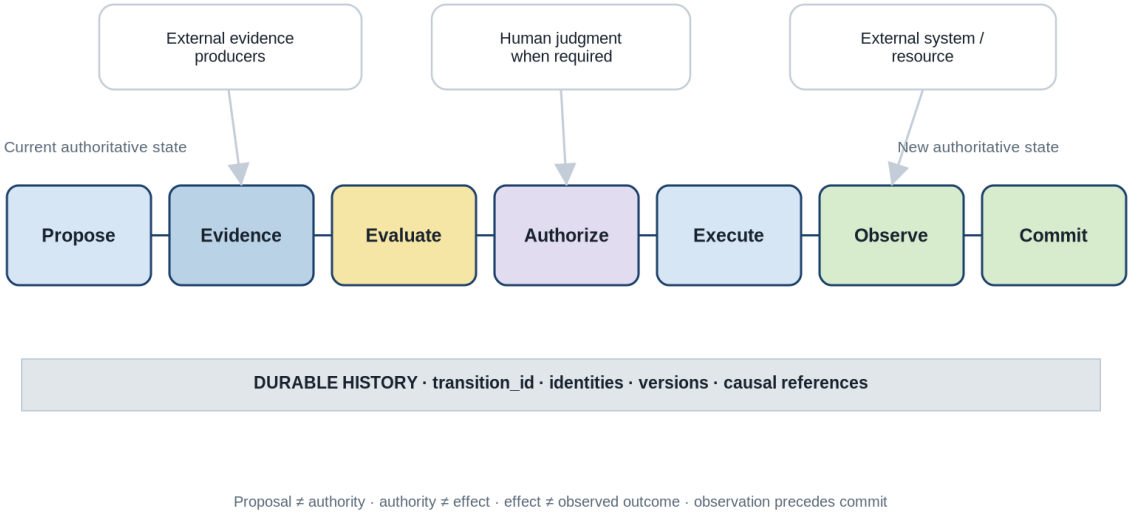
Here, Commit means recording authoritative post-state after reconciliation; it is unrelated to a source-control commit.

Each boundary matters.

Stage	Meaning
Propose	Prepare the candidate change or requested operation. No permission to make it consequential is implied.
Gather evidence	Collect durable observations or references about the object, environment, dependencies, identity, prior state, and other facts policy needs.
Evaluate	Apply a versioned rule to the current facts and requested operation.

Authorize	Create bounded authority specifying who may do what, to which resource, under which conditions, until when, and through which enforcement point.
Execute	Attempt the real side effect through the enforcement boundary.
Observe	Establish what became true in the external system; request acceptance or network success is not necessarily completion.
Commit	Record the new authoritative state only after the external result has been reconciled.

Figure 2. Anatomy of a consequential transition



This decomposition is deliberately stricter than a normal workflow edge. A workflow engine can encode all seven stages, but if the important relationships are represented only by ad hoc fields and callbacks, control of the transition still depends on application convention.

3. Requirements for Controlling Consequential Transitions

These requirements don’t assume a particular implementation.

3.1 Durable execution

R1 — Survive interruption without losing logical position.

A consequential transition may wait for minutes, weeks, or months. The execution system must recover from process crashes, machine loss, network failure, and ordinary restarts without silently skipping or repeating logical stages.

R2 — Give consequential transitions an explicit identity and lifecycle.

Important mutations need a stable place to attach evidence, policy decisions, authority, effects, observations, and failure semantics. If consequential mutations happen as scattered tool calls, there is no reliable control boundary.

R3 — Preserve a reconstructable execution history.

A later investigator must be able to recover the ordered events that led to the transition. Durable control flow is not enough if the causal evidence and decisions cannot be associated with it.

R4 — Make versions part of replay and interpretation.

The meaning of a historical decision can depend on the workflow definition, policy, evidence schema, and relevant software versions that were active at the time. The history must preserve enough identity to interpret or migrate old executions safely.

3.2 Trusted evidence

R5 — Persist or durably reference the evidence used for consequential decisions.

“Tests passed” is incomplete. The record needs the relevant artifact, test identity, environment, time, result, or a stable reference to an authoritative record containing them.

R6 — Preserve evidence provenance and producer identity.

The system must distinguish the actor’s assertion from an observation produced by another service. Evidence should identify who or what produced it and what object or state it describes.

R7 — Make security-relevant history integrity-verifiable at the chosen assurance level.

Not every installation requires a cryptographically signed event chain. But the assurance model should be explicit. The system should know what kind of tampering its history can and cannot detect.

R8 — Prevent the requester from manufacturing the complete sufficient evidence set.

The requester may trigger tests, propose artifacts, and supply claims. It must not be able to unilaterally set every fact that grants its next authority.

3.3 Authorization

R9 — Keep policy outside the acting agent's unilateral control.

"External" is a trust relationship, not necessarily a network hop. A co-located policy engine can still be logically distinct from the actor if the actor cannot rewrite the rule that grants it authority.

R10 — Represent resulting authority explicitly.

A transient Boolean is often too weak for a multi-stage process. A successful decision should produce an inspectable authority object with at least a principal, action, resource, object binding, evidence basis, policy basis, issue time, expiry, audience, replay rule, and revocation state.

R11 — Make authority narrow, conditional, expiring, audience-bound, replay-aware, and revocable.

Here, replay-aware means preventing unintended reuse or replay of a grant or consequential request. It is distinct from workflow replay in R4.

"Agent X is trusted" is not a useful runtime artifact. A safer form is closer to:

Principal P may perform action A on resource R under grant G until time T, through enforcement point E, provided conditions C still hold.

3.4 Controlled action

R12 — Route consequential side effects through an enforcement boundary that validates current authority.

A policy decision that can be bypassed is advisory. The system must know where the consequential effect becomes real and ensure that the required authority is checked there.

R13 — Represent human decisions as first-class events bound to the same transition.

A human decision should carry identity, scope, time, and a durable association with the exact transition. It should not live as an unrelated email, chat message, or generic “approved” flag.

R14 — Answer “Why was this action permitted, and what happened?”

Given a consequential action after the fact, the system should be able to reconstruct:

requester → request → pre-state → evidence → evidence producers → policy/version → decision → grant or human decision → enforcement attempt → external observation → post-state.

That chain is the integration test for the whole architecture.

4. See the delta before choosing the architecture

A team can own excellent components and still have weak transition semantics.

A common composed stack looks approximately like this:

Agent → Workflow → Policy call → Boolean → Application glue → API → Logs

Around that line sit other useful systems: test services, artifact stores, approval UIs, identity providers, monitoring systems, event stores, and ticketing or incident tools.

None of those components is the problem. The risk lies in the joins.

Who converts a test result into a typed fact?

Who decides that this producer is competent to establish that fact?

Which exact evidence set did the policy evaluate?

Which exact object version and pre-state did the decision apply to?

Who turns “allow” into bounded authority?

Where are expiry, audience, replay, and revocation defined?

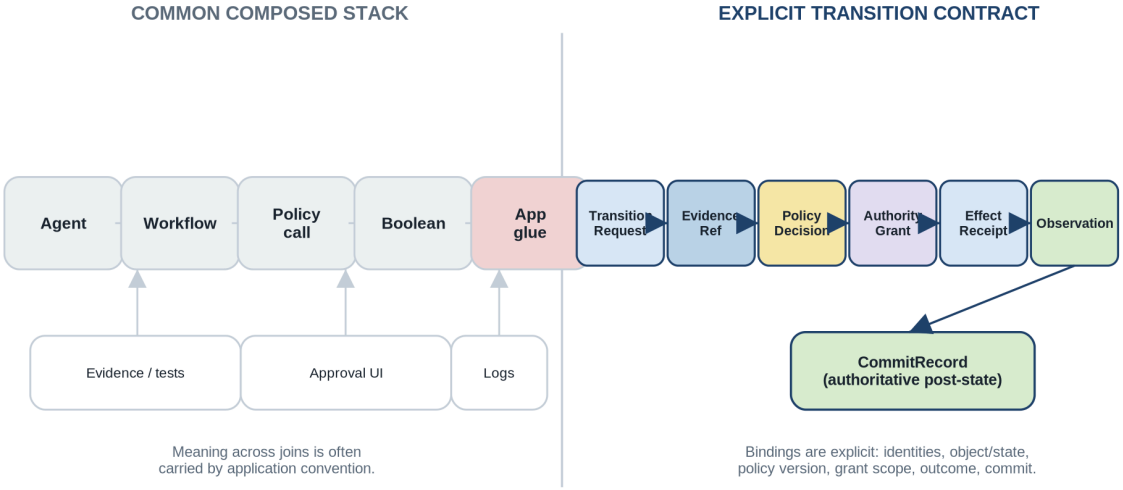
Where does the real API call validate the authority?

What happens when the target may have changed but the response was lost?

How is the observed outcome bound back to the request and grant?

How does an investigator later traverse the complete causal chain without guessing?

Figure 3. Current composition versus an explicit transition contract



The vulnerable part of many systems is not a missing product category. It is the custom semantic work between products:

- evidence normalization and competence mapping;
- binding the exact evidence set, pre-state, and policy version;
- converting PolicyDecision into AuthorityGrant;
- defining grant audience, expiry, replay, and revocation;
- validating the grant at the actual effect boundary;
- reconciling ambiguous outcomes before retry;
- binding Observation to CommitRecord; and
- preserving causal correlation across the whole transition.

That is the narrow engineering problem this paper addresses.

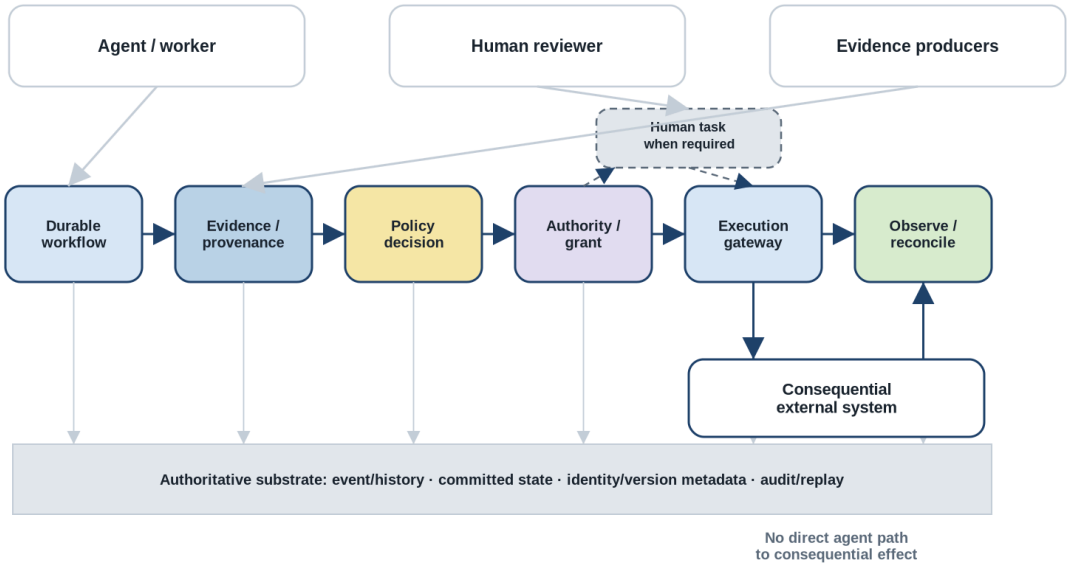
5. Reference architecture

The requirements and joins imply a small number of operational roles.

These are logical responsibilities, not mandatory deployment boundaries. An implementation may co-locate them, distribute them across existing systems, enforce authority directly at a target or capability boundary, or expose them through a gateway. The requirement is the semantic separation and enforceable binding, not a particular service topology.

Component	Responsibility
Agent or worker runtime	Investigates, proposes, writes, plans, or produces candidate changes. Its assertions are not authoritative by default.
Durable workflow engine	Owns long-lived control state, timers, retries, waits, branching, recovery, and transition lifecycle.
Evidence and provenance service	Resolves facts needed for a decision and preserves their origins across test, artifact, identity, attestation, database, or signed-history sources.
Policy decision engine	Evaluates a named policy version over the requested operation, current state, evidence, and relevant identities; returns the decision and decision metadata.
Authority service	Turns a successful policy result into a bounded grant binding transition, principal, action, resource/state, evidence and policy basis, expiry, audience, replay semantics, and revocation state.
Human task service	Creates a human decision task when required; the result is bound to the same transition.
Execution gateway	Validates the grant and relevant current state immediately before invoking the consequential external action.
Observation and reconciliation adapter	Establishes what became true; reconciles ambiguous outcomes before retry or commit.
Authoritative history and audit interface	Preserves the causal links and the committed post-state.

Figure 4. Reference architecture for controlling a consequential transition



6. Walk one deployment through a consequential transition

Consider an agent that proposes a patch to a production service. The purpose of the example is only to make the interfaces concrete.

Stage	Deployment example
Propose	Agent creates commit C and requests deployment of service S; workflow creates transition T and records requester, repository, commit, target environment, and intended operation.
Gather evidence	Controlled build produces artifact A; test and security services emit evidence with producer identity, artifact binding, time, and environment or dependency context. The agent may request these checks but cannot authoritatively assert their success.
Evaluate	Policy P17 evaluates whether requester X may deploy artifact A to service S using T, current target state, and the named evidence set.
Authorize or escalate	If sufficient, create grant G bound to T, principal, deploy action, service S, artifact A, policy P17, evidence E1…En, deployment-gateway audience, expiry Texp,

	single-use semantics, and active revocation state. Otherwise create a human task bound to T.
Execute	Execution gateway validates G against principal, resource, artifact, target state, expiry, audience, replay rule, revocation state, and execution-time conditions; records target operation identifier O when available.
Observe	Observation layer establishes deployed version and relevant target state. Ambiguous outcomes remain reconciling rather than being assumed successful.
Commit	Workflow records the new authoritative state and durable references from T through evidence, policy, grant, effect, and observation.

A useful consequence of this decomposition is that historical truth and current authority can diverge safely. A test result can remain true as a past observation while no longer authorizing execution after a dependency change.

7. Existing primitives are already strong

Durable workflow, policy, provenance, and event history already exist.

Primitive	Representative systems	What is already available
Durable workflow	Temporal; Dapr Workflow; AWS Step Functions Standard; Camunda; LangGraph [2–13]	Long-lived control flow, recovery, replay, callbacks, checkpoints, and human waits.
Externalized policy	OPA; Cedar [14–17]	Policy decisions separated from acting application logic, with structured authorization inputs and decision metadata.
Durable history and provenance	KurrentDB; in-toto; Sigstore [18–20]	Event streams, concurrency control, identity-bound provenance, attestations, and transparency mechanisms.
Human workflow	Camunda; Step Functions callback patterns; LangGraph interrupts [9,11–13]	First-class pauses and resumptions around human review or approval.

Dapr narrows the remaining gap even further. Its workflow stack now combines event-sourced execution with optional signed workflow history, SPIFFE workload identity, cross-application history propagation, completion attestations, and workflow scheduling access policy [4–7]. That is evidence that the ecosystem is already converging across categories.

Closely related work covers neighboring parts of the same transition. Schrock's Action Evidence Boundary focuses on executor-side evidence/action correlation, authorization, one-time authority, and reconciliation [21]. Wu et al.'s EBL-Core specifies an execution-boundary release/grant contract binding a canonical candidate, policies, evidence, and grant lifecycle [22]. Wadkins specifies what must be bound at decision time so authorization, enforcement, execution, and effect remain independently determinable afterward [23]. Together, these works cover overlapping executor-boundary mechanisms. This paper spans the full evidence → policy → authority → effect → observation → commit transition.

The remaining gap is narrower:

The problem is not lack of primitives. It is which relationships remain application convention.

Appendix A provides the detailed current tool analysis, coverage matrix, and product-to-architecture mapping.

8. The interface contract

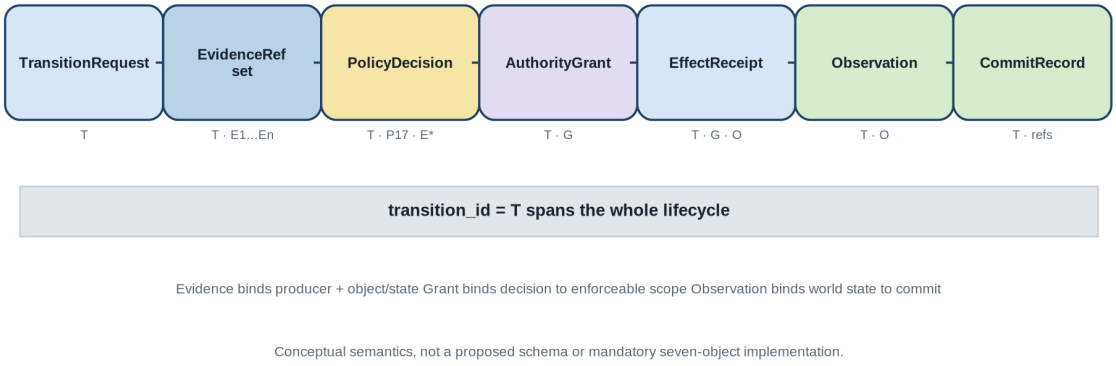
Calling the gap "integration" is too vague. A small set of conceptual objects makes it inspectable.

These names are illustrative. They are not a proposed standard, API, schema, or mandatory decomposition.

Object	Core fields	Role
TransitionRequest	transition_id; requester; action; resource; pre_state_ref	Identifies the consequential change. transition_id remains stable through evidence, authorization, execution, observation, and commit.
EvidenceRef	evidence_id; fact/type; producer_identity; object_binding; observed_at; integrity/provenance_ref; freshness/invalidation conditions	States what fact is asserted, who produced it, what object/state it describes, and when it stops being competent evidence.
PolicyDecision	transition_id; policy_id/version; evidence_set; pre_state_ref; decision; decision_metadata	Binds the authorization answer to the exact transition inputs.
AuthorityGrant	grant_id; transition_id; principal; action; resource/object_binding; evidence_basis; policy_basis; issued_at/expires_at; audience; conditions; replay_rule; revocation_state	Bridges policy and consequential effect with bounded, inspectable authority.

EffectReceipt	transition_id; grant_id; operation_id; target; attempted_at; acceptance/result metadata	Records what the execution boundary attempted and any durable operation identity; does not by itself prove final state.
Observation	transition_id; observer_identity; target; observed_state; observed_at; observation_integrity_ref	Records what the external system later established as true.
CommitRecord	transition_id; authoritative_post_state; request/evidence/decision/grant/effect/observation refs; committed_at	Closes the transition by binding authoritative post-state to its causal chain.

Figure 5. Interface objects across one transition



The useful review question is not “Did you implement these seven structs?”

It is:

Where do these semantics live, who can mutate them, and which bindings are enforced rather than inferred?

9. Three implementation forms

The transition contract can be realized in several forms.

No implementation form is assumed.

Form	Advantage	Tradeoff
Composition pattern	Can be built now from existing workflow, policy, evidence, identity, and enforcement components.	Critical semantics remain bespoke unless the team defines and enforces the bindings consistently.
Protocol or profile	Can standardize selected objects and bindings while preserving product choice and interoperability.	Requires agreement on semantics such as identity, freshness, revocation, object binding, replay, and outcome—not just a common JSON shape.
Integrated runtime	Can make the evidence-backed authorized transition native, simplifying replay, audit, developer experience, and end-to-end guarantees.	Broadens runtime responsibility across evidence, policy, authority, execution, observation, and state commitment.

10. What first-class support could look like

The following non-normative pseudocode illustrates the contract.

```
transition DeployChange {
   requester: agent
   resource: production/service-X

   require:
      signed_build_pass
      signed_test_pass
      current_security_result

   policy:
      deployment_policy_v17

   authorize:
      deploy(service-X, artifact)
      expires: 15 minutes
      single_use: true

   execute:
      deployment_gateway

   observe:
      production_version == artifact.version
```

```
    commit:
        service-X.current_version = artifact.version
}
```

The point is what the developer declares.

The developer declares the consequential transition.

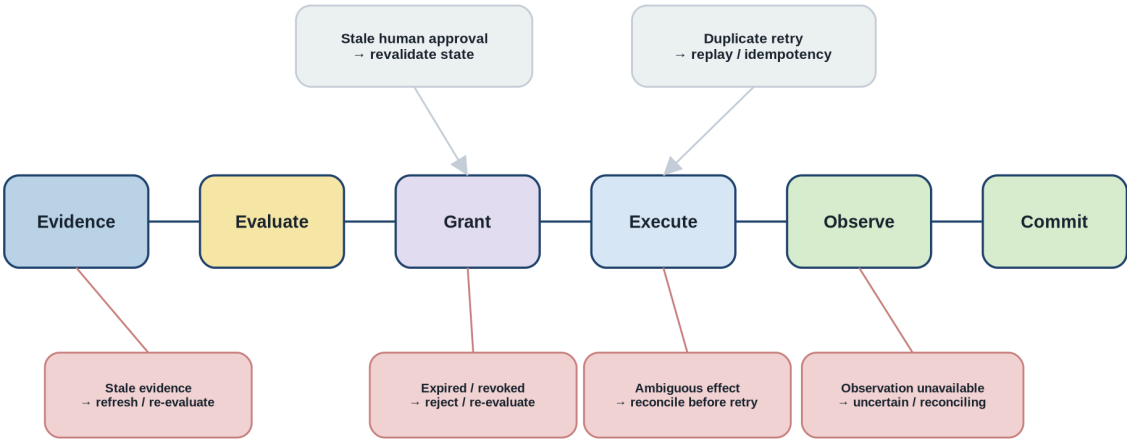
The runtime or composed system takes responsibility for preserving the evidence binding, policy version, grant identity, expiry, audience, enforcement attempt, operation identity, observation, and commit relationship.

Existing components can still sit behind each interface.

11. Failure semantics as acceptance tests

A reference architecture becomes real when the happy path fails.

Figure 6. Failure and reconciliation map



The transition contract is defined as much by what it refuses to commit under uncertainty as by its normal path.

Failure case	Expected behavior	Architecture question
Stale evidence before authorization	Keep the historical observation; deny or refresh evidence before authorization.	Can the system distinguish “this happened” from “this is still competent evidence”?
Evidence invalidated after grant issuance	Make the grant unusable or fail execution-time conditions.	How does evidence invalidation propagate into current authority?
Policy changes after evidence collection	Record the applicable policy version; make survival of any existing grant explicit.	Which policy version applies to the transition now?
Grant expires before use	Reject at the execution boundary; do not silently renew.	Is expiry enforced where the effect becomes real?
Grant is revoked	Enforcement point observes revocation within the declared risk window.	What is worst-case revocation propagation time, and can another path bypass it?
Side effect may have occurred but response is lost	Reconcile external state or use an idempotent operation identity before retry.	What prevents uncertainty from becoming a duplicate effect?
Execution succeeds but observation fails	Remain uncertain/reconciling; do not commit a guessed post-state.	Where is uncertainty represented, and what event resolves it?
Human approval arrives after state drift	Keep approval as historical evidence, but revalidate execution against current state.	What exactly did the person approve?
Workflow or schema version changes during a long-running transition	Preserve compatibility, migrate deliberately, or block; keep replay/version semantics explicit.	Which version interprets the historical transition?
Duplicate retry reaches the execution boundary	Apply the declared replay rule; single-use grants and operation IDs must not become multi-use.	Where is duplicate detection authoritative?

If an implementation cannot say what remains authoritative and what transition is legal in these cases, the transition contract is incomplete even if every component is individually reliable.

12. Minimum viable transition contract

A useful first implementation can be small.

It needs:

• a durable workflow engine;
• an append-oriented or otherwise auditable history;

• identifiable evidence producers and durable evidence references;
• an external policy decision point;
• an explicit scoped authority representation;
• an execution gateway for selected consequential effects;
• a human-task primitive where needed;
• an outcome-observation and reconciliation mechanism; and
• a queryable causal audit trail.

The minimum semantic integration is smaller still:

1. The transition has a stable identity.
2. Evidence has producer identity and object/state binding.
3. Policy evaluation names the evidence set, pre-state, and policy version.
4. Successful evaluation creates an explicit grant.
5. The real execution boundary validates and consumes that grant.
6. The external result is observed and tied back to the transition.
7. The complete relation is durable and queryable.

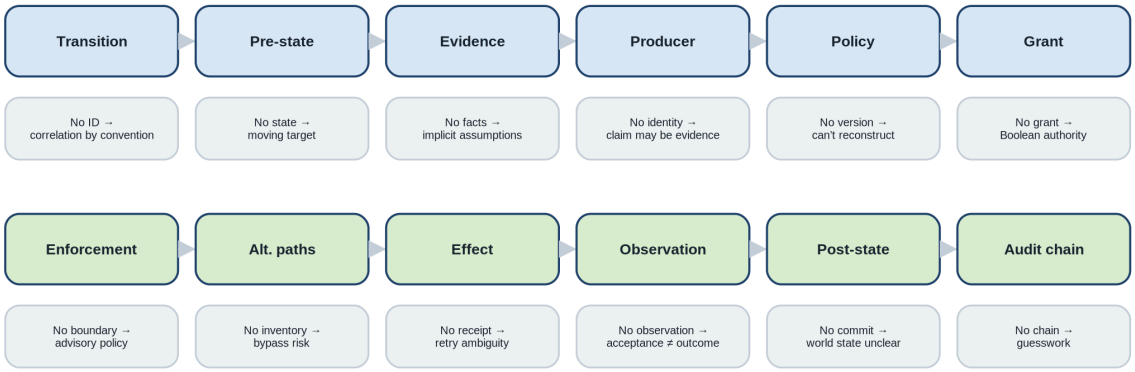
Everything else can initially be adapters.

The architecture should also permit weaker assurance profiles. Not every spelling correction needs signed evidence and a revocable grant. The obligation is that consequential boundaries have explicit semantics rather than hidden convention.

13. Architecture-review diagnostic

The same contract can be used directly in a design review.

Figure 7. Architecture-review diagnostic map



Use the map to distinguish native semantics, external semantics, and application convention.

Use the table below as a diagnostic during design review.

Review item	If missing
Consequential transition — What changes externally, and what stable transition identity follows the change?	Audit correlation depends on workflow-local context, timestamps, or naming convention.
Authoritative pre-state — Which object version, environment state, dependency epoch, account state, or other starting condition is the request about?	Policy and approval may be applied to a moving target.
Required evidence — What facts must be true before the action may become real?	“Approved” may encode an undocumented bundle of assumptions.
Evidence producer — Who or what produced each material evidence item, and can the beneficiary forge, suppress, replace, or reinterpret it?	Actor claims may be functioning as evidence.
Policy version — Which exact rule and data produced the authorization decision?	A historical allow/deny cannot be reproduced or interpreted reliably.

Authority object — Is there an explicit grant, or only a Boolean copied into application state?	Scope and lifetime are implicit.
Grant constraints — What are the scope, expiry, audience, replay rule, and revocation path?	Authority can outlive the facts that justified it.
Enforcement point — Is the check on the path where the consequential effect becomes real?	The policy decision may be advisory.
Alternate effect paths — Can direct credentials, administrator APIs, emergency mechanisms, legacy interfaces, or alternate services bypass the enforcement boundary?	The enforcement story is incomplete.
Ambiguous-result handling — What happens when the request may have succeeded but no definitive response returned?	Retries can duplicate effects or the workflow can commit false state.
Outcome proof — Is request acceptance treated as completion, or is target state actually observed?	Workflow state may be mistaken for external reality.
Authoritative post-state — What does the system finally commit as true, and what causal references support it?	The system cannot say when the transition actually completed.
Audit reconstruction — Can an investigator traverse request → evidence → policy → grant → effect → observation → commit without guessing?	The system is observable but not reconstructable.

14. Conclusion

The tooling problem is smaller than it first looks.

Durable execution, policy decision, provenance, human tasks, event history, and agent checkpointing already have strong implementations. Some systems are converging across several of these capabilities.

What remains is the join.

For a consequential transition, the execution contract should durably connect:

evidence → policy → authority → effect → observed result → committed state.

Today that relationship can be built from existing systems. In the systems reviewed here, we did not find one runtime that treats the complete relationship itself as the primary object and enforces its semantics end to end.

That doesn’t imply the answer is another platform.

The transition contract makes that relationship explicit and enforceable. It can be realized as a composition pattern, a protocol/profile, or an integrated runtime.

Its usefulness is testable without adopting any particular implementation. An architect should be able to identify the transition, inspect the evidence and policy bindings, find the grant, locate the real enforcement point, reconcile the external outcome, and reconstruct the committed state afterward.

If those relationships are explicit, enforced, and reconstructable, the system can control consequential transitions even when the required components come from different systems.

If those relationships exist only in convention and glue, that is the part of the architecture that still needs engineering.

Appendix A — Current tool coverage

This appendix summarizes current tool coverage relevant to the requirements above. It is not a product ranking or a market survey.

A.1 Temporal

Temporal provides one of the clearest implementations of durable control flow. A Workflow Execution can run for long periods and recover after failures. Temporal records Commands and Events in an Event History; its documentation describes that history as the ordered source of truth used to replay workflow code back to prior logical state [2,3]. External interactions are placed in Activities, whose results become events rather than being recomputed during replay [2].

For this architecture, Temporal strongly covers durable execution and history and provides useful machinery for explicit transitions, version-aware long-running control, external side-effect calls, and later reconstruction.

Temporal documentation also describes Principal Attribution as a pre-release capability that can stamp authenticated caller identity onto Event History events [3], moving part of producer identity closer to the workflow substrate.

What Temporal does not, by itself, make first-class is the complete evidence-to-grant relation. Application code can store evidence references, call a policy service, represent a grant, and put

enforcement into an Activity or gateway. Those semantics remain composed rather than being the native unit of the workflow engine.

A.2 Dapr Workflow

Dapr is strong evidence that trustworthy workflow history is already available.

Dapr Workflows are event-sourced and reconstruct state from history [4]. With WorkflowHistorySigning enabled, Dapr signs workflow-history batches with the sidecar's SPIFFE X.509 identity, chains the hashes, persists certificates with the history, and verifies the chain before workflow execution continues [5].

Current documentation also describes cross-identity completion attestations for child workflows and activities so completions from a different SPIFFE identity carry their own proof [5].

Dapr 1.18 adds workflow-history propagation across workflows and applications. A receiver can inspect a parent's own history or full lineage, including the applications and workflow instances that contributed to the chain; with signing enabled, propagated chunks can be cryptographically verified [6].

Dapr also has WorkflowAccessPolicy. The callee sidecar can restrict which SPIFFE-identified applications may schedule named workflows or activities. The current resource is an allow-list, and the documented operation today is scheduling [7].

These features cover significant portions of durable execution, reconstructable history, provenance, integrity, and cross-application identity.

The remaining distinction is precise. WorkflowAccessPolicy answers whether a caller may schedule named work. It is not documented as a transient grant derived from a particular evidence set, policy version, object state, expiry, replay rule, and revocation condition. Signed history can be inspected by downstream code, but the runtime does not surface the complete evidence → policy → scoped grant → enforced effect → observed result relation as one first-class transition object.

Dapr demonstrates convergence and narrows the gap.

A.3 AWS Step Functions

AWS Step Functions provides explicit state-machine execution, service integrations, and durable Standard workflows. AWS describes Standard workflows as long-running, durable,

auditable, and exactly-once at the workflow level unless retries are configured; execution state persists between state transitions and execution history is available for a defined retention period [8].

State machines can call services, wait for jobs, or pause for callbacks [9,10].

The callback task-token pattern is relevant to human gates. AWS explicitly lists human approval as a case where a Standard workflow can pause until an external process returns the token [9].

Step Functions covers durable orchestration and human waits well, and service integrations provide concrete effect points.

The task token is primarily a continuation mechanism, not a general evidence-backed authority object. Evidence provenance, external policy, grant semantics, and the causal binding between evidence and effect remain application responsibilities.

The distinction between Standard and Express workflows also matters: Standard workflows persist execution state and record histories in Step Functions, while Express workflows use different execution semantics and rely on CloudWatch logging for execution history [8].

A.4 Camunda

Camunda is especially strong where many agent frameworks remain weak: a human task is a modeled process object rather than an out-of-band pause.

In Camunda 8, when a process reaches a user task, Zeebe creates a user-task instance and the process waits until the task is completed [11]. User tasks support assignment, scheduling, forms, lifecycle events, and authorization around human work.

Camunda also provides durable process orchestration and job workers for automated work.

The missing semantics are not basic orchestration. The reviewed user-task and worker model does not itself define a durable authority grant whose validity is bound to independently sourced evidence and enforced at a separate consequential side-effect boundary. Those relationships can be modeled and implemented with services, but they remain composition.

A.5 LangGraph

LangGraph demonstrates how agent runtimes are moving toward ordinary workflow requirements. Its persistence layer stores thread state through checkpointers, supporting

continuity, fault tolerance, time travel, and human-in-the-loop execution [12]. Its interrupt mechanism saves graph state, waits for external input, and resumes the same thread later [13].

Interrupts can be placed immediately before sensitive tool calls, allowing a person to approve, reject, or edit a proposed operation [13].

That is useful coverage for durable agent-native execution and human intervention.

The trust model is different from the transition contract defined here. A checkpoint is durable application state, not automatically competent or independently produced evidence. The graph can carry evidence references and invoke external policy, but the framework does not by itself separate the actor from the sufficient evidence set, mint a scoped grant, or provide an independently enforced effect boundary.

A.6 OPA and Cedar

OPA is explicitly designed as a Policy Decision Point. Applications or Policy Enforcement Points send structured input; OPA evaluates policy and data and returns decisions. OPA documentation recommends placing the engine near enforcement points and supports policy distribution and decision logging [14,15].

Cedar provides a complementary model. An authorization request is structured around principal, action, resource, and context. The authorizer evaluates permit and forbid policies and returns Allow or Deny plus diagnostics, including determining policies [16,17].

Both systems make authorization explicit and keep it outside business logic.

Neither is intended to be the durable workflow, evidence store, grant service, effect executor, or observation system. In their documented core interfaces, the output is an authorization decision rather than the complete scoped, expiring, revocable grant plus effect and observation relation described in this paper.

A.7 KurrentDB

KurrentDB, formerly EventStoreDB, provides durable streams of events with stream revisions and optimistic concurrency. An append can require that the stream still be at a particular revision; if another writer has changed it, the append fails [18].

Newer KurrentDB versions also support atomic multi-stream append operations with expected versions across streams [18].

This is strong machinery for durable history and the Commit stage, especially when authoritative state is event-sourced.

An event store does not establish that an observation is semantically trustworthy merely because it is durable. Producer identity, evidence competence, policy evaluation, grant semantics, and effect enforcement remain separate concerns.

A.8 in-toto and Sigstore

in-toto shows how explicit provenance can work in a concrete domain. A signed layout defines supply-chain steps and authorized functionaries. Link metadata records commands, materials, products, and the functionary performing a step. Verification checks that expected authorized steps occurred and artifact rules hold [19].

Sigstore provides a complementary identity and transparency mechanism. A signer can obtain a short-lived certificate tied to an identity, sign an artifact, and record the signature, certificate, and artifact digest in Rekor's append-only transparency log. Verification can check the artifact signature, expected identity, trust root, and transparency-log inclusion [20].

These systems strongly inform evidence provenance and integrity. They are not general workflow engines or policy-to-authority runtimes.

A.9 Requirements coverage matrix

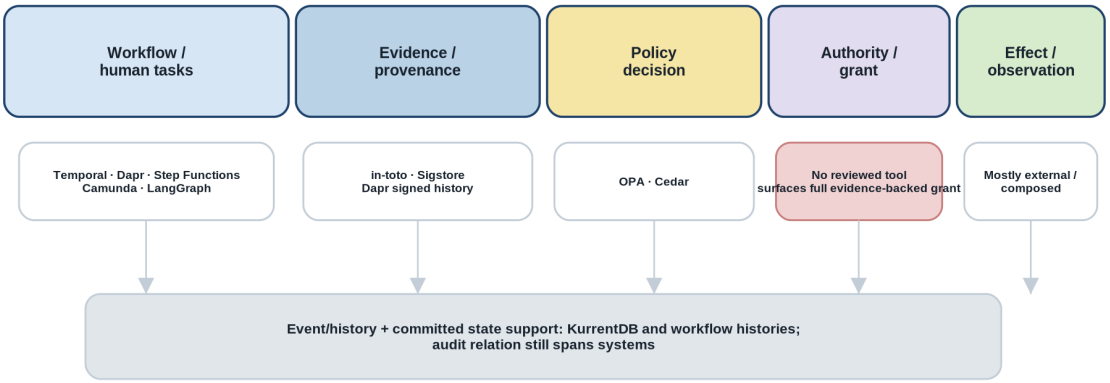
Figure A1. Requirements coverage matrix

System	Exec	History	Evidence	Policy	Grant	Human	Effect	Observe	Audit
Temporal	Native	Native	Partial	External	External	External	External	External	Partial
Dapr Workflow	Native	Native	Partial	Partial	Not provided	External	Partial	Partial	Partial
AWS Step Functions	Native	Native	External	External	External	Partial	Partial	Partial	Partial
Camunda 8	Native	Native	External	External	External	Native	Partial	External	Partial
LangGraph	Native	Partial	External	External	External	Native	Partial	External	Partial
OPA	N/A	Partial	External	Native	Not provided	N/A	External	N/A	Partial
Cedar	N/A	External	External	Native	Not provided	N/A	External	N/A	Partial
KurrentDB	N/A	Native	Partial	N/A	N/A	N/A	N/A	N/A	Partial
in-toto / Sigstore	N/A	Partial	Native	External	N/A	N/A	N/A	N/A	Partial

Native = first-class documented capability · Partial = useful machinery, incomplete transition-contract semantics · External = normal composition expected

A.10 Where reviewed tools fit in the reference architecture

Figure A2. Reviewed tools mapped onto the reference architecture



The ecosystem is rich in primitives. The sparse area is the explicit end-to-end binding across the joins.

References

[1] Brad Pierce, “Agents Act. Systems Remember. Rules Authorize. People Govern.: A Practical Architecture for Governed AI Agents,” Zenodo, 2026.

[2] Temporal Documentation, “Temporal Workflow.”

[3] Temporal Documentation, “Events and Event History.”

[4] Dapr Documentation, “Workflow Protocol — State & History” and “Workflow overview.”

[5] Dapr Documentation, “Workflow history signing.”

[6] Dapr Documentation, “Workflow history propagation.”

[7] Dapr Documentation, “WorkflowAccessPolicy spec.”

[8] AWS Step Functions Developer Guide, “Choosing workflow type in Step Functions.”

[9] AWS Step Functions Developer Guide, “Discover service integration patterns in Step Functions.”

[10] AWS Step Functions API Reference, “GetExecutionHistory.”

[11] Camunda 8 Documentation, “User tasks” and “User task authorization.”

[12] LangChain Documentation, “LangGraph Persistence.”

[13] LangChain Documentation, “LangGraph Interrupts.”

[14] Open Policy Agent Documentation, “How to Deploy OPA.”

[15] Open Policy Agent Documentation, “External Data” and “OPA Management APIs and Architecture.”

[16] Cedar Policy Language Reference Guide, “How Cedar authorization works.”

[17] Cedar Policy Language Reference Guide, “Best practice: Using the context” and “What is Cedar?”

[18] KurrentDB Documentation, “Appending events” and KurrentDB v26.0 release notes.

[19] in-toto Documentation, “Getting started.”

[20] Sigstore Documentation, “Overview.”

[21] I. Schrock, “The Action Evidence Boundary for Consequential Agent Effects,” Work in Progress, Internet-Draft draft-schrock-action-evidence-boundary-05, August 2026.

[22] M. Wu, L. Wang, Y. Zhang, and J. Deng, “From Intent to Execution Grant: An Execution-Boundary Conformance Profile for High-Risk AI Actions,” arXiv:2609.11596, September 2026.

[23] D. Wadkins, “Independent Determinability of Agent Actions,” Work in Progress, Internet-Draft draft-wadkins-agentproto-action-determinability-00, September 2026.